
マルチスレッド化の具体的方法

演習パッケージ: P11_MT

Geant4 11.3.0準拠

Geant4講習会資料: HEP/NP/Space 2025



KEK

大学共同利用機関法人

高エネルギー加速器研究機構

本資料に関する注意

- 本資料の知的所有権は、高エネルギー加速器研究機構およびGeant4 collaborationが有します
- 以下のすべての条件を満たす場合に限り無料で利用することを許諾します
 - 学校、大学、公的研究機関等における教育および非軍事目的の研究開発のための利用であること
 - ・ Geant4の開発者はいかなる軍事関連目的へのGeant4の利用を拒否します
 - このページを含むすべてのページをオリジナルのまま利用すること
 - ・ 一部を抜き出して配布したり利用してはいけません
 - 誤字や間違いと疑われる点があれば報告する義務を負うこと
- 商業的な目的での利用、出版、電子ファイルの公開は許可なく行えません

演習の目標

1. マルチスレッド・モードで磁場およびSensitive Detectorを扱う場合、シリアル・モードとは異なる扱いが必要となる。
 - 磁場を扱う方法については、H06_PrimaryGeneratorで使った磁場中の e^+/e^- 発生プログラムのGeometry定義を変更してマルチスレッド・モードで動くようにした P11_MT/MultiThread_MagField のコードを読む。
 - Sensitive Detector については BGO_Two_Output をマルチスレッド化してP11_MT/MultiThread_Outputをコンパイルし、バッチモードで実行してスコアをCSVファイルに出力する。Jupyter labを用いて結果を可視化する。
2. マルチスレッド化の費用対効果費を考える。
 - P11_MT/MultiThread_Output と BGO_Two_Output を同じイベント数で実行して実行時間を比較した結果を参考にする。
 - マルチスレッドプログラミングの課題を理解する

目次

1. P11__MT/MultiThread__Magfield
 1. コードを読んでSerialとの違う部分を理解する
 2. 磁場中での荷電粒子の様子を可視化して見る
2. P11_MT/MT_BGO_Output
 1. P11_MT/BGO_Output を下敷きにしてMT対応としている
 2. コードを読んでマスターとワーカの違う部分を理解する
 3. ワーカごとにヒット集合を別のCSVファイルにOSのI/Oを使って書き出すことによってGeant4内部での衝突を回避している。衝突の問題について理解を深める。
 4. 実行してMTの実行時の振る舞いを理解する

演習の準備

マルチスレッド実行環境について

■ OSが提供する情報

- Windows: CTRL + Shift + ESC 同時押しでタスクマネージャーを出し、パフォーマンスを見るとコア数とメモリーが分かる
- MacOS
 - `% sysctl -n hw.physicalcpu_max`
 - `% sysctl -n hw.logicalcpu_max`
 - メモリーは「このMac」>「システムレポート」
- Linux
 - `% cat /proc/cpuinfo` CPU コア数
 - `% cat /proc/meminfo` 実装メモリー量

→ 端末を起動して確認しておく

■ 仮想マシンを立ち上げる前に設定できること

- VM2025.10 では起動前に環境設定でコア数とメモリー量を変えられる

マルチスレッド・モードを使う前の注意

- 今までのhands-onはGeant4をシリアル・モードに設定して、プログラムを動かしてきた
- この演習ではGeant4をマルチスレッド・モードに設定して演習を行う
- モードの切り替えは以下の手順で行う
 - hands-on開始時点では、受講者のhome directoryにある~/.bashrcに
export G4FORCE_RUN_MANAGER_TYPE='Serial'
が設定されており、これでシリアル・モードが選択されている
 - この環境変数をunsetすることでマルチスレッド・モードに変更される
unsetは各自のターミナルで以下のコマンドを実行する:

```
$ unset G4FORCE_RUN_MANAGER_TYPE
```

マルチスレッド・モードの演習を始める前に、
上記の'unset'を必ず行うこと！

MultiThread_MagField

マルチスレッド・モードでの磁場の扱い

MultiThread_MagFieldプログラムのファイル構造の確認

課題: 1 MultiThread_MagFieldディレクトリのファイル構造を確認する

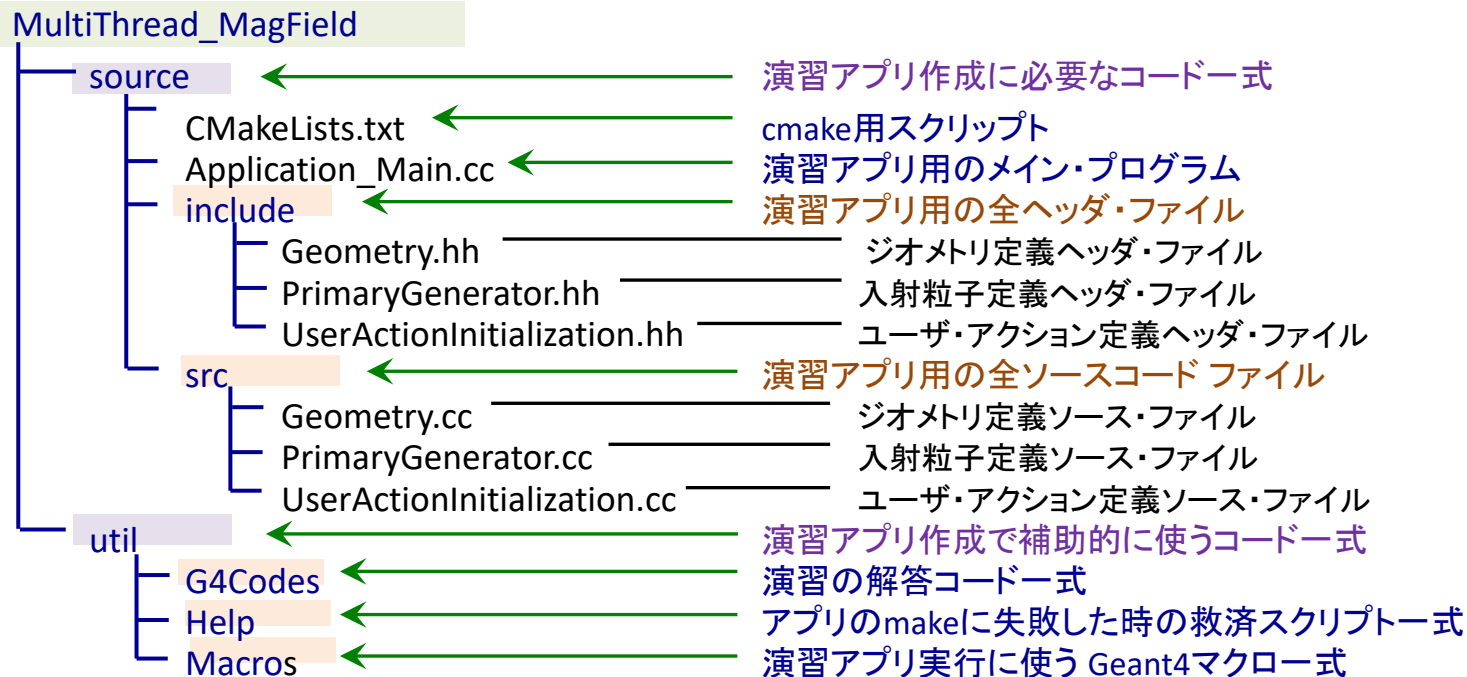
1) 以下に従って、'ls'コマンドを実行

```
$ cd ~/Gean4Tutorial_2025/P11_MT
$ ls MultiThread_MagField
```

[メモ]

1. コマンド入力には必ずtcsh補完機能を使う
2. スライドのコマンドを「コピペ」するのは危険

2) MultiThread_MagFieldのファイル構造が下記であることを確認



MultiThread_MagFieldプログラムの概要

■ 演習プログラムの目的

- マルチスレッド・モードを使用する場合、磁場の記述はシリアル・モードとは異なり、Geometry.hhに新しい関数' ConstructSDandField()' を定義し、この関数の中に磁場定義を記述しなければならない
- H06_PrimaryGeneratorで使った磁場中のe+/e-発生プログラムのGeometry定義部分と、マルチスレッド・モード用に用意したGeometry.定義部分を比較してその相違を理解する

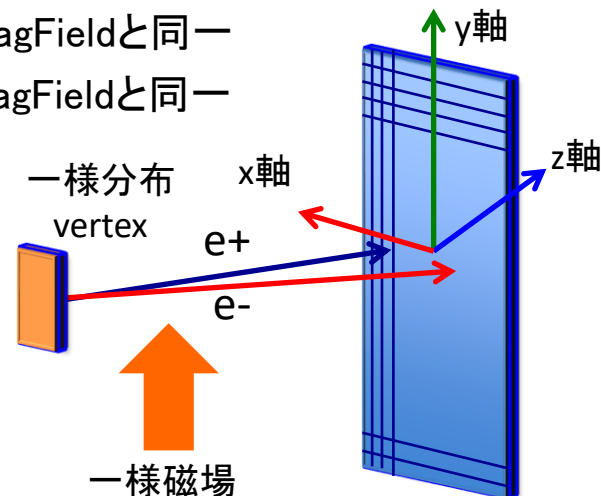
[注] この演習では、用意されているプログラムを編集することなく、そのまま使う

■ プログラムの構成

- mainプログラム: P06_PrimaryGenerator: MyGen_MagFieldと同一
- ジオメトリ: P06_PrimaryGenerator: MyGen_MagFieldをMultiThread用に変更
- PhysicsList: P06_PrimaryGenerator: MyGen_MagFieldと同一
- PrimaryGenerator: P06_PrimaryGenerator: MyGen_MagFieldと同一

■ 組み込まれているジオメトリと粒子発生機能

- ジオメトリ: Pixel_One検出器と一様磁場
- 粒子発生: 平面上に一様分布するvertexから同時に2粒子を発生



MultiThread_MagField: Geometry.hhの記述

マルチスレッド・モードを使用する場合、Geometry.hhに新しい関数

ConstructSDandField()

を定義し、この関数の中に磁場定義を記述しなければならない

1) MultiThreadモードとSerialモードでのGeometry.hhの違いを理解する

```
$ colordiff -y --width=160 ../../util/G4Codes/Geometry.hh_MagField Geometry.hh
```

[注] ../../util/G4Codes/Geometry.hhはP06_PrimaryGenerator: MyGen_MagFieldで使ったGeometry.hhと同じもの

2) colordiffの結果

Serial用	MultiThread用
<pre>//+++++ // Geometry.hh //+++++ #ifndef Geometry_h #define Geometry_h 1 #include "G4UserDetectorConstruction.hh" class G4VPhysicalVolume; //----- class Geometry : public G4UserDetectorConstruction //----- { public: Geometry(); ~Geometry() override; G4VPhysicalVolume* Construct() override; }; #endif</pre>	<pre>//+++++ // Geometry.hh //+++++ #ifndef Geometry_h #define Geometry_h 1 #include "G4UserDetectorConstruction.hh" > class G4VPhysicalVolume; > class G4GlobalMagFieldMessenger; //----- class Geometry : public G4UserDetectorConstruction //----- { public: Geometry(); ~Geometry() override; G4VPhysicalVolume* Construct() override; void ConstructSDandField() override; private: static G4ThreadLocal G4GlobalMagFieldMessenger* fMagFieldMessenger; }; #endif</pre>

この関数を新たに追加する

← 関数の実装は次のスライド参照

MultiThread_MagField: Geometry.ccの記述

Geometry.hhで導入した新しい関数ConstructSDandField()のに一様磁場を定義する

1) MultiThreadモードとSerialモードでのGeometry.ccの違いを理解する

```
$ colordiff -y --width=160 ../../util/G4Codes/Geometry.cc_MagField Geometry.cc
```

[注] ../../util/G4Codes/Geometry.hhはP06_PrimaryGenerator: MyGen_MagFieldで使ったGeometry.hhと同じもの

2) colordiffの結果

Serial用	MultiThread用
===== 同一のinclude文を省略 =====	
#include "Geometry.hh"	#include "Geometry.hh"
#include "G4GlobalMagFieldMessenger.hh"	#include "G4GlobalMagFieldMessenger.hh"
	> #include "G4AutoDelete.hh"
	> //-- Global objects -----
	> G4ThreadLocal G4GlobalMagFieldMessenger* Geometry::fMagFieldMessenger = nullptr;
	> //-----
===== 同一のgeometry定義文を省略 =====	
// Add uniform magnetic field to the world auto magFieldVector = G4ThreeVector{ 0.0, 1.0*tesla, 0.0 }; new G4GlobalMagFieldMessenger{ magFieldVector };	< < < <
// Return the physical world return physVol_World; }	// Return the physical world return physVol_World; }
	>
	> //-----
	> void Geometry::ConstructSDandField() > //-----
	> {
	> // Add uniform magnetic field to the world
	> auto fieldValue = G4ThreeVector{ 0.0, 1.0*tesla, 0.0 };
	> fMagFieldMessenger = new G4GlobalMagFieldMessenger(fieldValue);
	> fMagFieldMessenger->SetVerboseLevel(1);
	>
	> // Register the field messenger for deleting
	> G4AutoDelete::Register(fMagFieldMessenger);
	> }

MultiThread_MagFieldのビルドと実行(今回は省略)

課題: MultiThread_MagFieldをビルドして実行する

1) MultiThread_MagFieldのディレクトリに移行する:

```
$ cd ~/Gean4Tutorial_2024/P11_MT/MultiThread_MagField
```

CBDir (Current Base Dir)とよぶ

2) アプリをビルドして実行: [注] 必要なファイルは全てディレクトリに用意されている

```
$ mkdir build
$ cd build
$ cmake ../source
$ make
$ make install
```

Build dirの作成

cmake実行

まとめてもOK

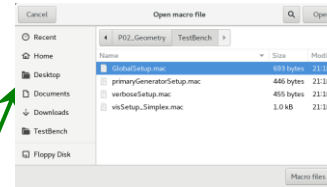
\$ make install

```
$ cd ..
$ mkdir TestBench
$ cd TestBench
$ cp ../util/Macros/* .
$ ../bin/Application_Main
```

TestBenchの作成

用意されている
macrosをコピー

アプリを実行



GlobalSetup.macを選び
'Open'をクリック

3) Qt画面のコマンド窓に以下のコマンドを入力実行

```
/control/execute GlobalSetup.mac
/run/beamOn 50
```

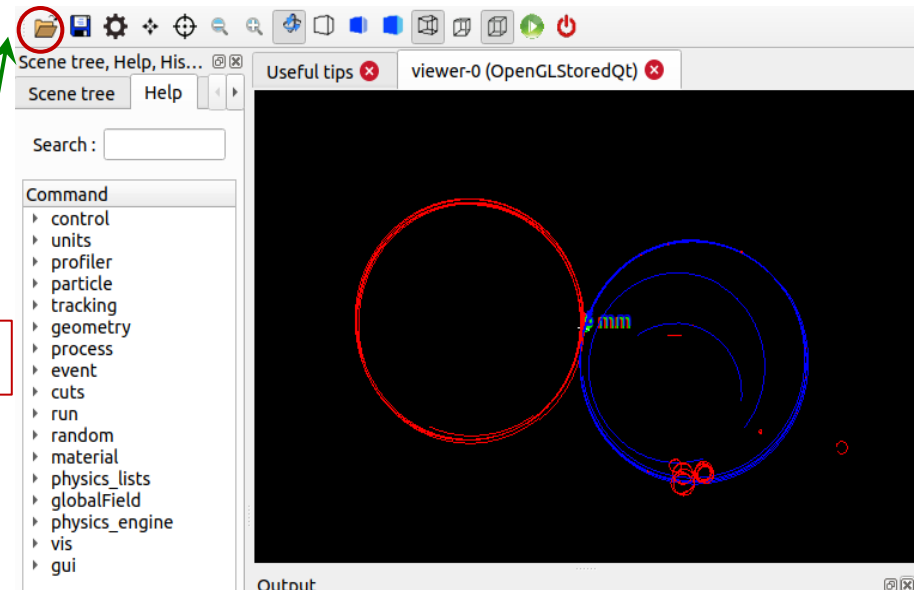
ここをクリックしてマクロ
ファイル実行も可能

- 右図の様にgenantinoの飛跡が見えることを確認

4) Qtコマンド窓に以下のコマンドを入力し、アプリを終了

```
exit
```

[注] 'exit'は先頭に'/ 'が不要



MultiThread_MagFieldの実行結果

前課題の50 events処理の結果、以下の様な出力が得られる

```
=====
--> G4TaskRunManager::CreateAndStartWorkers() --> Creating 7 tasks with 7 events/task...
=====

Adding task 0 to task-group...
Adding task 1 to task-group...
Adding task 2 to task-group...
Adding task 3 to task-group...
Adding task 4 to task-group...
Adding task 5 to task-group...
Adding task 6 to task-group...
Adding task 7 to task-group...
G4WT1 > ### Run 0 starts on worker thread 1.
G4WT2 > ### Run 0 starts on worker thread 2.
G4WT0 > ### Run 0 starts on worker thread 0.
G4WT3 > ### Run 0 starts on worker thread 3.
G4WT2 > [thread 2] Thread-local run terminated.
G4WT3 > [thread 3] Thread-local run terminated.
G4WT1 > [thread 1] Thread-local run terminated.
G4WT0 > [thread 0] Thread-local run terminated.
G4WT2 > [thread 2] Run Summary
G4WT2 > [thread 2]   Number of events processed : 8
G4WT2 > [thread 2]   User=0.520000s Real=0.231446s Sys=0.030000s [Cpu=237.6%]
G4WT3 > [thread 3] Run Summary
G4WT3 > [thread 3]   Number of events processed : 0
G4WT3 > [thread 3]   User=0.000000s Real=0.000282s Sys=0.000000s [Cpu=0.0%]
G4WT1 > [thread 1] Run Summary
G4WT1 > [thread 1]   Number of events processed : 28
G4WT0 > [thread 0] Run Summary
G4WT0 > [thread 0]   Number of events processed : 14
G4WT0 > [thread 0]   User=1.080000s Real=0.503483s Sys=0.080000s [Cpu=230.4%]
G4WT1 > [thread 1]   User=1.440000s Real=0.671972s Sys=0.120000s [Cpu=232.2%]
Run terminated.
Run Summary
  Number of events processed : 50
  User=2.050000s Real=1.238679s Sys=0.180000s [Cpu=180.0%]
```

50 eventsを8個のタスクに振り分ける

タスクをこの4個のスレッドで処理する
[注] この例では4 coresのmachineを使用

thread#2: 8 events処理

thread#3: 0 events処理

thread#1: 28 events処理

thread#0: 14 events処理

マルチスレッド・モードでのtracking verbose出力

tracking verboseを‘1’にしてプログラムを走らせると以下の様になる



同時に走っているスレッドからの出力競合で非常に読みにくなる

[注] tracking verboseを‘1’以上にすると処理速度が極端に遅くなるので、できるだけ避けること

```
G4WT0 > *****
G4WT0 > * G4Track Information: Particle = e-, Track ID = 370, Parent ID = 2
G4WT0 > *****
G4WT0 >
G4WT0 > Step#    X(mm)    Y(mm)    Z(mm) KinE(MeV)  dE(MeV) StepLeng TrackLeng  NextVolume ProcName
G4WT0 > 0      -250     752     29.3   0.0033      0      0      0 PhysVol_World initStep
G4WT0 > 1      -250     752     29.2    0      0.0033   0.37   0.37 PhysVol_World eIoni
G4WT2 > 0      -308     463     37.3   0.00145    0      0      0 PhysVol_World initStep
G4WT2 > 1      -308     463     37.3    0   0.00145  0.1182  0.0982 PhysVol_World eIoni
G4WT3 > 1     -82.3    -65.6    -155    0   0.0028   0.28   0.28 PhysVol_World eIoni
G4WT3 >
G4WT1 >
G4WT2 >
G4WT3 > *****
G4WT0 > *****
G4WT1 > *****
G4WT1 > * G4Track Information: Particle = e-, Track ID = 104, Parent ID = 2
G4WT3 > * G4Track Information: Particle = e-, Track ID = 41, Parent ID = 2
G4WT0 > *****
G4WT0 > * G4Track Information: Particle = e-, Track ID = 369, Parent ID = 2
G4WT0 > *****
G4WT0 >
G4WT0 > Step#    X(mm)    Y(mm)    Z(mm) KinE(MeV)  dE(MeV) StepLeng TrackLeng  NextVolume ProcName
G4WT0 > 0      -204     744     82.6   0.0021      0      0      0 PhysVol_World initStep
G4WT0 > 1      -204     744     82.6    0   0.0021   0.175   0.175 PhysVol_World eIoni
G4WT2 > *****
G4WT2 > * G4Track Information: Particle = e-, Track ID = 139, Parent ID = 2
G4WT3 > *****
G4WT0 > *****
G4WT1 > *****
G4WT1 >
G4WT1 > Step#    X(mm)    Y(mm)    Z(mm) KinE(MeV)  dE(MeV) StepLeng TrackLeng  NextVolume ProcName
G4WT0 > *****
G4WT0 > * G4Track Information: Particle = e-, Track ID = 368, Parent ID = 2
G4WT0 > *****
G4WT1 > 0      -320     308     -44   0.00237    0      0      0 PhysVol_World initStep
G4WT2 > *****
G4WT2 >
```

MT_BGO_Output

マルチスレッド・モードでのスコアリング出力

スコアリングと結果の出力をするマルチスレッドプログラム

- 先の例はグローバルなジオメトリ情報をワーカが使えるようにするため、ConstructSDAndField() を使わねばならないことを知った。
- それ以外にもG4VUserRunAction を継承して、マスタースレッドとワーカースレッドのユーザアクションを実装せねばならない。
- ワーカーのスレッドが実行するG4VSensitiveSetector, G4UserSteppingAction, G4UserEventActionなどのクラスの実装において、共有資源の競合、相互排除、Critical Sectionの正しい指定など、並列プログラミングにつきもののデバッグ困難な問題に備えなければならない。
- そこで、BGO_Output を出発点にして、そのマルチスレッド化を行う。
 - **メインプログラム**で各ワーカに**それぞれ**データ保管庫 SimData を用意する
 - SensitiveVolume ワーカは**互いに独立に**そこにスコアを書き込む
 - ランの終わりには**マスター**ランアクションがワーカのデータを**逐次的**に取り出し、CSVファイルに書き出す。

マルチスレッド： マスターとワーカー別のクラス

マスター：一つのインスタンス

○印は新規作成成分

Main()

G4VUserDetectorConstruction
Construct()

G4VUserPhysicsList

○G4VUserActionInitialization
BuildForMaster()

○ConstructSDAndField()

読むだけ

Build()

読むだけ

各ワーカーに一つのインスタンス

- G4VUserPrimaryGeneratorAction
- ○ G4UserRunAction
- G4UserEventAction
- G4UserSteppingAction
- ○ G4VSensitiveDetector

Geant4 MTの動き方

■ マスタースレッドは最初に

- グローバルなジオメトリと物理リストの初期化
- マスターとワーカ別にユーザアクションの設定
- ユーザインターフェイスの用意
- ワークスレッドを起動して
- ワーカイベント数を分配する
- 終わりにワーカの算出した結果の取りまとめ

■ ワーカは互いに独立にシミュレーションを実行する

- シミュレーション実行中は最低限の相互排除を行うことがありうる
- 計算結果をマスターに渡す

■ MT_BGO_Outputは BGO_Output を元に最小限の変更でマルチスレッド対応にしたものである。以下、マルチスレッド対応のところを見てゆく。

手順： データストアはワーカ別に用意する

- メインプログラムでsimdataを用意して Geometry, SensitiveVolume, UserInitialization, RunActionにポインタを教えるのは BGO_Two_Outputと同じ。

```
// Set SimData pointer

auto simdata = new SimData[nvec]; // ワーカーのスコアデータ用配列

auto geometry = new Geometry();
geometry-> SetSimData(simdata);


// Job termination
delete [] simdata;           // 自分で配列をデリートする
delete visManager;
delete runManager;

return 0;
```

手順: メインプログラムでやること

- Main プログラム

1. runManager は既定値、つまりMulti-thread

G4RunManagerFactory::CreateRunManager(G4RunmanagerType::Default);

2. Thread 数(既定は2)の設定には環境変数または

runManager -> SetNumberOfThreads(NUMBER_OF_THREAD);

- ✓ 後で必要な時にワーカー数を求めるには

auto nworkers= G4RunManager::GetRunManager()-> GetNumberOfThreads();

- ✓ 以降のクラスの中でスレッドの情報を得るには

- ✓ G4bool IsMaster() でマスタースレッドかどうかを判定する

- ✓ ワーカーが自分のスレッド番号を得るには

- ✓ #include “G4Threading.hh” して

- ✓ G4int threadID = G4Threading::G4GetThreadId();

- マスターは -1 () マイナス1

- ワーカーは 0から始まる整数値

メインプログラムから抜粋

■ Application_Main.cc

- 4 threads に決め打ちにしておこう。増減はお好みで

```
// Construct the default run manager
G4int nvec = 4;           // スレッド数
auto* runManager =
    G4RunManagerFactory::CreateRunManager(G4RunManagerType::Default);
runManager -> SetNumberOfThreads(nvec);

// Set up mandatory user initialization: Geometry
// Set SimData pointer
auto simdata = new SimData[nvec]; // SimDataを nvec 要素の配列にする
auto geometry = new Geometry();
geometry-> SetSimData(simdata);    // Geometry が SimData の場所を知る
runManager-> SetUserInitialization(geometry);
```

途中略

```
delete [] simdata;           // 自分の責任で
```

手順: ユーザアクションでやること

- G4UserRunAction ではマスターとワーカのアクションをそれぞれ記述する。
 - `IsMaster() == true` で判定してマスターのユーザアクションを記述
 - `Void UserActionInitialization::BuildForMaster() const`
 - `auto runaction = new RunAction();`
 - `SetUserAction( runaction );`
 - `//etc.`
- ワーカには
 - `IsMaster() != true` のときは
 - `Void UserActionInitialization::Build() const`
 - `auto runaction = new RunAction();`
 - `SetUserAction( runaction );`
 - `SetUserAction(new PrimaryGenerator() );`
 - `//etc.`

RunAction が自分の役割を識別するためにIsMaster()を使う

ユーザアクション コード抜粋

■ UserActionInitialization

```
//-----  
void UserActionInitialization::Build() const  
//----- ワーカ  
{ SetUserAction( new PrimaryGenerator() );  
  auto runaction = new RunAction();  
  runaction -> SetSimData(simdata);  
  SetUserAction(runaction);}  
// ----- マスター  
void UserActionInitialization::BuildForMaster() const  
{ auto runaction = new RunAction();  
  runaction-> SetSimData(simdata);  
  SetUserAction(runaction);}
```

手順: ジオメトリでやること

- グローバルなジオメトリを記述するのはConstruct()。
 - World volume を返すのはこちら。
- ワーカーが使えるようにスレッドローカルなSD (Sensitive Detector) と磁場 (Field)はConstructSDandField()を使わねばならない。
 - Field のコード例はMultiThread_MagneticField で示した。
 - Multithred_Output は共有が関わらないのもっと簡単である。

```
// Return the physical volume of 'World'
return physVol_World;
} // Construct() ends here

// Sensitive volume
void Geometry::ConstructSDandField()
{
    SensitiveVolume* aSV = new SensitiveVolume("SensitiveVolume");
    aSV->SetSimData(simdata);
    //assign it to SDManager
    auto SDman = G4SDManager::GetSDMpointer();
    SDman->AddNewDetector(aSV);
    SetSensitiveDetector("LogVol_BGO", aSV);
}
```

手順: RunAction ではマスターとワーカで仕事が違う

```
void RunAction::BeginOfRunAction(const G4Run*)
{
    if ( IsMaster() != true ) { // ワーカは自分のデータストアを初期化する
        G4int tid = G4Threading::G4GetThreadId();
        simdata[tid].Initialize();
    }
}

void RunAction::EndOfRunAction(const G4Run*)
{
    if ( IsMaster() ) { // マスターはランの終わりにワーカのデータストアを集めて出力する
        G4String fName = "P11_MT.csv";
        std::ofstream outputfile(fName);
        auto nworkers= G4RunManager::GetRunManager()-> GetNumberOfThreads();
        for(auto i = 0; i < nworkers; i++) {
            auto record = simdata[i].GetEdepRecord(); // 順次集荷するループなので衝突無し
            for (auto val : record) {
                outputfile << std::get<0>(val); // エネルギー
                outputfile << " , " ; // カンマ区切り
                outputfile << std::get<1>(val); // ステップ長
                outputfile << std::endl ; } // CSV 出力
            }
        outputfile.close();
    }
}
```

SimData.hh

BGO_Two_Output (“Serial” 版) と同じ。

```
class SimData {
public :
    SimData() = default;
    ~SimData() = default;
    SimData(const SimData&) = delete;
    void operator = (const SimData&) = delete;

    void Fill(G4double eval, G4double sval);
    void Initialize(); const std::vector<std::tuple<G4double, G4double>>& GetRecord() const;
private :
    std::vector<std::tuple<G4double, G4double>> Record;
};

inline void SimData::Initialize(){
    Record.clear();
}
inline void SimData::Fill(G4double eval, G4double sval ){
    Record.emplace_back(eval, sval);
}
inline const std::vector<std::tuple<G4double, G4double>>& SimData::GetRecord() const{
    return Record;
}
```

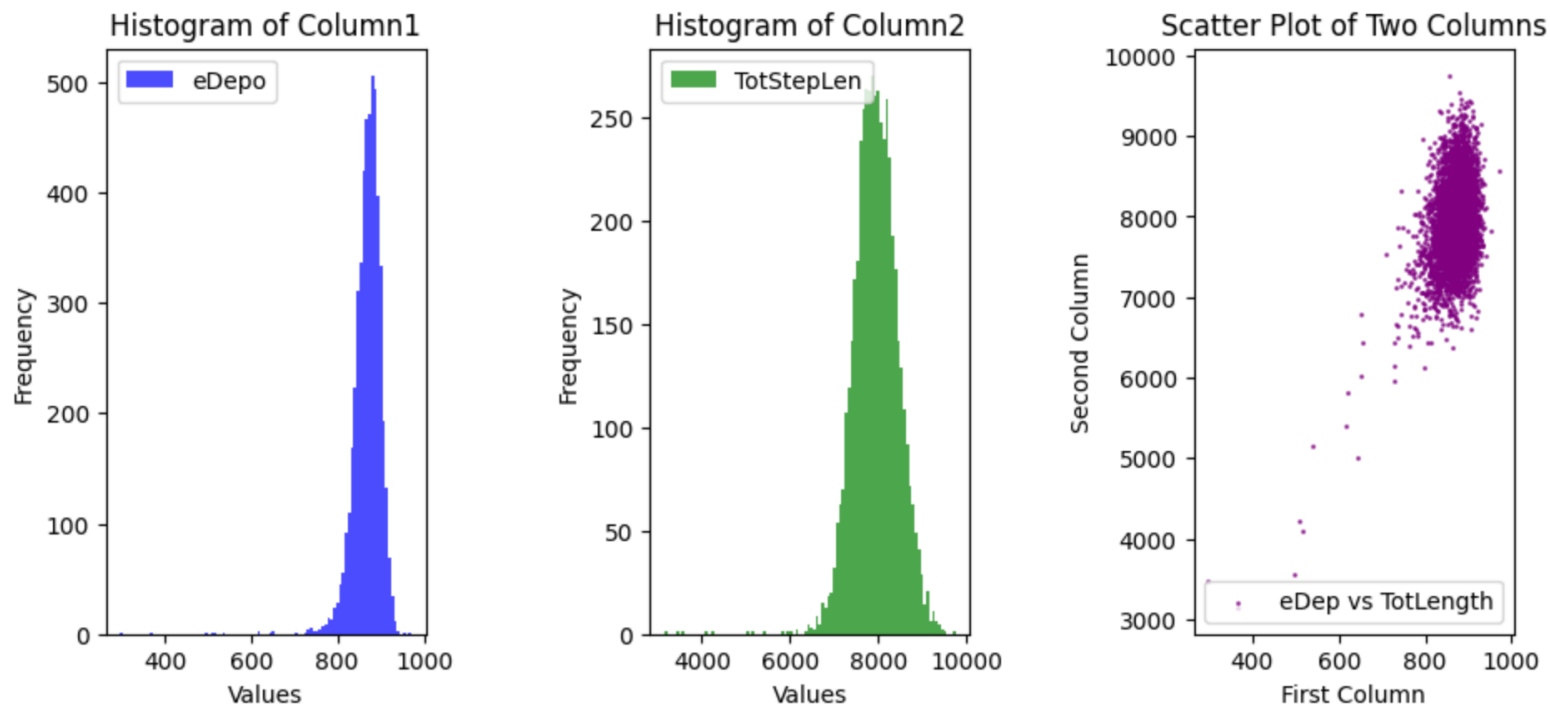
MultiThread_Out でのワーカの作法

- ワーカはそもそも共有資源を使わないのが最良の方策
 - Global 変数を使わない
 - Static 変数を使わない
- マスターRunActionはワーカーのデータを順次集めてCSVファイルに書き出した
 - ワーカがそれぞれ一つのファイルを書き出すときは、そのバッファーや割り込み管理はOSが行うので安全だが、ここでは使わない。
- G4cout はスレッド安全だが std::cout はそうでない。デバッグにG4coutは使える。
- 衝突の可能性があるのでファイルの入力は使わないようにするのが吉。

コンパイルして実行

```
G4WT0 > ##### Run 0 starts on worker thread 0.
G4WT2 > [thread 2] Thread-local run terminated.
G4WT0 > [thread 0] Thread-local run terminated.
G4WT0 > [thread 0] Run Summary
G4WT3 > [thread 3] Thread-local run terminated.
G4WT2 > [thread 2] Run Summary
G4WT1 > [thread 1] Thread-local run terminated.
G4WT1 > [thread 1] Run Summary
G4WT3 > [thread 3] Run Summary
G4WT2 > [thread 2]   Number of events processed : 19934
G4WT0 > [thread 0]   Number of events processed : 20022
G4WT1 > [thread 1]   Number of events processed : 20022
G4WT3 > [thread 3]   Number of events processed : 20022
G4WT2 > [thread 2]   User=1.940000s Real=0.655598s Sys=0.000000s [Cpu=295.9%]
G4WT0 > [thread 0]   User=5.390000s Real=1.493711s Sys=0.000000s [Cpu=360.8%]
G4WT1 > [thread 1]   User=3.800000s Real=1.108049s Sys=0.000000s [Cpu=342.9%]
G4WT3 > [thread 3]   User=4.210000s Real=1.210614s Sys=0.000000s [Cpu=347.8%]
Run terminated.
Run Summary
  Number of events processed : 80000
  User=306.520000s Real=76.415788s Sys=0.080000s [Cpu=401.2%]
```

electron1GeV.mac 5000 evnts 実行結果



P11_MT.csv から
シャワーエネルギーと荷電粒子のヒストグラムと
両者の相関プロット

BGO の輻射長は約 1cm 二つのBGO でほぼ 20 rad. Len.

マルチスレッドでどれだけ速くなるか

■ ハンズオンで簡単な実験

- MT対応のMT_BGO_Output にマクロbenchmark.macを渡して4 threads 実行して時間を測る
- “Serial” のBGO_Outputも同じマクロを与えて実行時間を測る
- Ryzen 8 cores/ 16 threads のWindows Subsystem for Linuxで実行し時間はrun 終了時に出力されたもの
- 2回繰り返し実行したがほぼ同じ結果で、MTはReal time で3.35倍速い

	MT_BGO_Output	BGO_Output
events	80000	80000
process	4	1
Real sec	76.5	256.6
User sec	308.9	257.5
Sys sec	0.1	0.1

(参考) マルチスレッドプログラムのデバッグ

- 並列化しないプログラミングとマルチスレッドプログラミングは別物といえるほどで、そのデバッグには困難なことが多い。
 - 終わりまで動作しても実行ごとに結果が異なることがある
 - システムの負荷状態で結果が異なることがある。
 - 並列化しない「正しい」ソースコードプログラムを持っていることが重要。
 - ・ マルチスレッドプログラミングでは気づかぬうちに問題点を入れてしまうことがある。並列化しない「正しい」プログラムと比較することでデバッグを助けられる
 - マルチスレッドの不具合の検出を自分で行わず安心できるツールに任せる。そもそもデバッガーは対象となるプログラムと並行して走るのだから道具を選ぶ。途中経過をstd::coutしても、いつも同じ結果とは限らない。
 - 並列化しない“Serial”で困らないのならば、無理にマルチスレッドにしない方がよいだろう。

演習 終了

評価例

- グスタフソンの式 仮定： 並列計算でのデータセットと同じデータセットをシリアル実行できると仮定して、並列実行時間の割りあい s がシリアル実行に費やされるものとする。
 - MT のスピードアップをシーケンシャル計算時間とマルチスレッド計算時間の比とすると $\text{speedup} = 139.97/40.1 = 3.49$ となる。
 - Knudsen speedupの式 $\text{speedup} = p + (1 - p) * s = 3.49$ ($p = 4$ スレッド数) とすると シーケンシャル部分の割りあい s は $s = 0.17$ となる。
 - MT 時間は $40.10 * (1 - 0.17) = 33.3 \text{ s}$, となるので延並列計算時間は $33.3 * 4 = 133.2 \text{ s}$ これにシーケンシャル時間 $40.1 * 0.17 = 6.82 \text{ s}$ を加えると 140 s となる。